



preparatic 31

TEMA 126

LA SEGURIDAD EN EL NIVEL DE APLICACIÓN.
TIPOS DE ATAQUES Y PROTECCIÓN DE
SERVICIOS WEB, BASES DE DATOS E
INTERFACES DE USUARIO.

Versión	31.1
Fecha de actualización	21/02/2026

Resumen Express

1 La seguridad a nivel de aplicación

Es necesario contemplar la seguridad en el desarrollo de aplicaciones desde el comienzo de los proyectos.

2 Seguridad en interfaces de usuario

La interfaz de usuario es el elemento de interacción entre el usuario y el sistema, y es preciso contemplar cuestiones de seguridad en su definición.

- **Autenticación:** proceso por el que el usuario acredite su identidad mediante unas credenciales
- **Autorización:** proceso por el que el sistema comprueba que un usuario tiene permiso para realizar la operación que ha solicitado.
- **Gestión de excepciones:** es conveniente no proporcionar al usuario información interna del sistema que un usuario malintencionado pueda aprovechar para atacar.
- **Validación de datos de entrada:** puede provocar que, a través de la introducción de datos inválidos, un atacante interactúe con un sistema de forma no deseada.
- **Trazabilidad y auditorías:** todas las capas de un sistema deben guardar información de excepciones o errores que permitan trazar su origen y corregir posibles defectos del software.

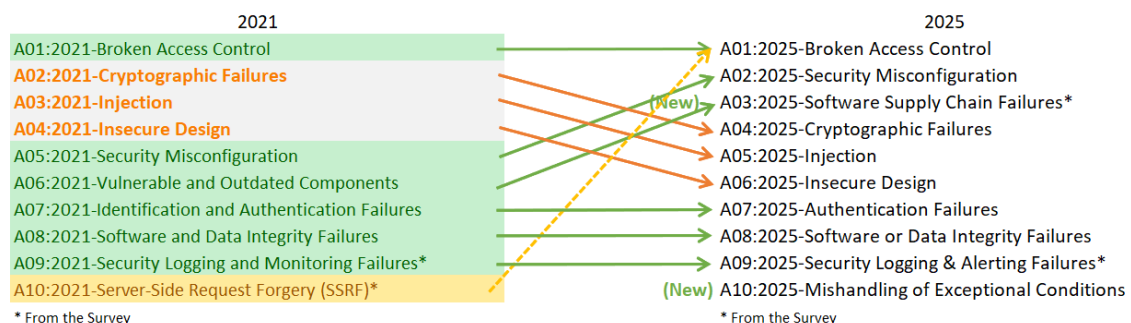
3 Seguridad en aplicaciones web

3.1 OWASP (Open Web Application Security Project)

Comunidad de seguridad informática que publica y actualiza cada 3 años el proyecto **OWASP Top 10** que es un documento donde se recogen los diez riesgos de seguridad más importantes en aplicaciones Web.

3.2 Amenazas y ataques en aplicaciones web

Los diez riesgos más críticos en Aplicaciones Web según OWASP Top 10 - 2025 son:



Hay dos nuevas categorías y una consolidación en el Top Ten para 2025:

- A01:2025 – Pérdida de control de acceso mantiene su posición en el #1 como el riesgo de seguridad de la aplicación más grave. Como se indica por la línea discontinua en la figura anterior, la falsificación de solicitud del lado del servidor (SSRF) se ha introducido en esta categoría.
- A02:2025 - La configuración de seguridad incorrecta de seguridad subió del #5 en 2021 al #2 en 2025. Las configuraciones erróneas son más frecuentes en los datos de este ciclo. El

3,00% de las aplicaciones probadas tenían uno o más de los 16 CWE de esta categoría. Esto no es sorprendente, ya que la ingeniería de software continúa aumentando la cantidad de comportamiento de una aplicación que se basa en configuraciones.

- A03:2025 – Uso de componentes con vulnerabilidades conocidas es una expansión de A06:2021- Componentes vulnerables y obsoletos para incluir un alcance más amplio de compromisos que ocurren dentro o a través de todo el ecosistema de dependencias de software, sistemas de construcción e infraestructura de distribución.
- A04:2025 - Los fallos criptográficos (exposición de datos sensible) caen dos lugares del #2 al #4 en el ranking. El punto clave es la criptografía utilizada o la falta de esta, tanto en datos almacenados como en tránsito.
- A05:2025 - La inyección cae dos puntos del #3 al #5 en el ranking, manteniendo su posición en relación con las fallas criptográficas y el diseño inseguro. La inyección incluye una variedad de problemas, desde secuencias de comandos entre sitios (alta frecuencia/bajo impacto) hasta vulnerabilidades de inyección SQL (baja frecuencia/alto impacto).
- A06:2025 – Diseño inseguro desliza dos lugares del #4 al #6 en el ranking como Configuración errónea de seguridad y fallas de la cadena de suministro de software saltaron. Esta categoría se introdujo en 2021.
- A07:2025 - Los fallos de autenticación. Esta categoría sigue siendo importante, pero el mayor uso de marcos estandarizados para la autenticación parece estar teniendo efectos beneficiosos sobre los casos de fallos de autenticación.
- A08:2025 - Los fallos de software o de integridad de datos continúan en el #8 de la lista. Esta categoría se centra en la falta de mantenimiento de los límites de confianza y verificar la integridad del software, el código y los artefactos de datos a un nivel más bajo que los fallos de la cadena de suministro de software.
- A09:2025 - El registro de seguridad y las fallas de alertas mantienen su posición en el #9. Esta categoría tiene un ligero cambio de nombre para enfatizar la importancia de la funcionalidad de alerta necesaria para inducir la acción apropiada en eventos de registro relevantes.
- A10:2025 – Manejo incorrecto de condiciones excepcionales es una nueva categoría para 2025. Esta categoría se centra en el manejo incorrecto de errores, errores lógicos, fallas abiertas y otros escenarios relacionados derivados de condiciones anormales que los sistemas pueden encontrar.

3.2.1 Otros ataques

- Desplazamientos por directorios (*path traversal*).
- *Cross-Site Tracing* (XST).
- Cross-Site Request Forgery (CSRF).
- Remote file inclusion (RFI) y Local File Inclusion (LFI).
- Ataques contra SSL/TLS.
- Fallos en el manejo de memoria.
- Confianza inapropiada en código generado por IA (Vibe coding).

3.3 Metodologías de seguridad en aplicaciones web

Una metodología de seguridad en aplicaciones web debe tener en cuenta lo siguiente:

- La seguridad en el ciclo de vida de desarrollo.
- Realizar pruebas de caja negra.

- Realizar pruebas de caja blanca.
- Implementar firewalls de nivel de aplicación (Web Application Firewalls, WAF).

OWASP ha desarrollado un marco de trabajo abierto denominado **SAMM** (*Software Assurance Maturity Model*) que pretende ayudar a las organizaciones en el proceso de formulación y aplicación de estrategias de seguridad en el software. El CCN-CERT ha elaborado una [guía de seguridad para los entornos y las aplicaciones Web: CCN-STIC-812](#).

4 Seguridad en servicios web

Tanto si se implementa la API con servicios SOAP o servicios REST, existen medidas que se pueden tomar para fortalecer la seguridad:

- Utilizar tokens. Configurar identidades confiables y controle el acceso a los servicios y a los recursos utilizando los tokens asignados a dichas identidades. Usar tecnologías de autenticación y control de acceso como JWT enmarcado en el framework OAuth 2.0 y API Keys, así como la expiración de tokens para limitar su tiempo de uso y evitar accesos prolongados en caso de compromiso.
- Utilizar métodos de cifrado y firmas. Cifrar los datos mediante un método como TLS. Solicitar el uso de firmas para asegurar que solamente los usuarios adecuados descifren y modifiquen sus datos.
- Identificar vulnerabilidades, por ejemplo, utilizando un escáner web de vulnerabilidades como OWASP ZAP. Mantener actualizados sus elementos de API, sus controladores, redes y su sistema operativo. Mantenerse al tanto de cómo funciona todo en conjunto, e identificar las debilidades que se podrían utilizar para entrar en sus API. Utilizar analizadores de protocolos para detectar los problemas de seguridad y rastrear las pérdidas de datos.
- Utilizar cupos y límites. Establecer un cupo en la frecuencia con la que se puede recurrir a su API, y dar seguimiento a su historial de uso. Encontrar más solicitudes a una API puede indicar un abuso. También podría ser un error de programación, como una solicitud a la API en un bucle sin fin. Establecer reglas de limitación para proteger sus API de ataques de denegación de servicio y picos de uso.
- Utilizar una puerta de enlace de API. Las puertas de enlace de API funcionan como el principal punto de control para el tráfico de las API. Una buena puerta de enlace le permitirá autenticar el tráfico, así como controlar y analizar cómo se utilizan sus API.
- La gestión de datos es clave para evitar la exposición innecesaria de información en las APIs. Limitar los datos que una API devuelve es fundamental para reducir el riesgo de filtraciones, y esto se puede lograr mediante *field filtering* (selección de campos), donde los clientes solo reciben la información estrictamente necesaria.

4.1 WS-Security

WS-Security (WSS): extensión al estándar SOAP para proporcionar mecanismos de seguridad a los mensajes SOAP. Proporciona autenticación, integridad y confidencialidad en el intercambio.

Especifica cómo usar tecnologías y estándares como aserciones SAML o un certificado X.509, XML Encryption y XML DSignture. Mediante la lectura de las extensiones WSS de cabecera del mensaje SOAP se identifica qué mecanismos ha escogido el emisor de este para autenticar al usuario final, cifrar el mensaje y firmarlo y de esta forma proceder a su validación.

4.1.1 Seguridad a nivel de transporte (TLS) y a nivel de mensaje (WSS)

Diferencias entre los dos mecanismos:

TLS	WSS
Extremo a extremo de nivel de transporte (IP+puerto)	Extremo a extremo de nivel de aplicación.
Credenciales de autenticación de navegador y servidor únicos sin intermediarios.	Credenciales de autenticación de usuario final. Permite múltiples credenciales de autenticación.
No es flexible: se aplica a todo el payload de igual manera durante toda la sesión	Muy flexible: puede aplicarse sólo a una parte del mensaje y discriminar entre Requests y Responses
Se sitúa entre el nivel TCP y el de aplicación	La misma seguridad se puede aplicar en diferentes tecnologías de transporte.

4.1.2 Tecnologías de autenticación, confidencialidad e integridad soportadas por WSS

Tecnologías soportadas:

- **Integridad:** WSS emplea **XML Signature** (W3C) para la firma de los mensajes SOAP. Un mensaje SOAP puede llevar ninguna, una o múltiples firmas XML Signature.
- **Confidencialidad:** se basa en la especificación **XML-Encryption** (W3C) que al igual que ocurre con XML Signature permite encriptar parte o la totalidad del mensaje.
- **Autenticación:** WS-Security utiliza extensiones XML en la cabecera denominadas *tokens de seguridad*. Ejemplos de tokens disponibles: **username/password**, certificados **X.509**, tickets **Kerberos**, aserciones **SAML**, tokens XrML, tokens XCBF o referencias a URIs. El uso de SAML permite también intercambiar aserciones de autorización para usuarios del servicio.

4.1.3 Otras extensiones para web services

WS-Security se usa únicamente para autenticación, confidencialidad (cifrado) e integridad (firma). Existen otras extensiones SOAP **para el ámbito de la seguridad**:

- **WS-Policy:** permite a un servicio web especificar sus requisitos de calidad de servicio y de seguridad para que puedan ser consultadas antes de invocar la interfaz WS. Las extensiones para especificar la tecnología aceptada de seguridad se denominan **WS-SecurityPolicy** y permiten definir qué tokens de autenticación se admiten, qué partes deben estar encriptadas, qué partes firmadas etc.
- **WS-Addressing:** proporciona mecanismos neutrales, independientes del protocolo de transporte, para direccionar mensajes y servicios web. Convierte los mensajes SOAP en unidades autónomas de comunicación
- **WS-Trust:** define cómo intercambiar tokens de seguridad con credenciales de acceso a servicios web de distintos dominios de confianza.
- **WS-Federation.** Una Federación es un conjunto de dominios de seguridad que han acordado compartir recursos de forma segura. Está soportado por las extensiones ofrecidas por WS-Security (seguridad en el mensaje SOAP), WS-Trust (intercambio de tokens de seguridad entre distintos dominios de seguridad), and WS-SecurityPolicy (publicidad de la política de seguridad).
- **WS-SecureConversation:** Extensión a WS-Security para el establecimiento de contextos de seguridad para el intercambio de una serie de mensajes SOAP durante toda la sesión que se establezca entre partes de un Web-Service (lo que se denomina una “conversación”). Se diferencia de WS-Security en que ésta norma protege cada uno de los mensajes SOAP de forma independiente, lo que introduce un elevado overhead, mientras que WS-SecureConversation cubre toda la conversación.

4.1.4 Crítica a WSS y alternativas

Principal crítica realizada a WS sobre SOAP y sus extensiones WSS: el overhead que introducen y su lentitud. Como alternativa, los servicios RESTful que son más ligeros y su despliegue suele ser más rápido. Ofrecen interfaces directas HTTP, los mensajes van directamente encapsulados en el payload de este protocolo. Su seguridad suele estar garantizada mediante TLS.

4.2 SAML

SAML (*Security Assertion Markup Language*) es un lenguaje de marcado desarrollado por el consorcio OASIS para el intercambio de información de autenticación y autorización entre proveedores de servicios de identidad, proveedores de servicios y usuarios de los servicios. Permite el establecimiento de identidades portables, entendidas éstas como la extensión de las operaciones de autenticación y autorización de un dominio de seguridad a otro distinto con el que el primero mantiene una relación de confianza. La última versión es la 2.0 y data de 2005.

El uso más extendido de SAML es como mecanismo de Single Sign On y de federación de identidades pero su alcance va más allá de la mera autenticación permitiendo intercambiar atributos de los usuarios de los servicios con el fin de establecer perfiles para procesos de autorización.

Características generales

- **Múltiples tecnologías de transporte:** Mecanismos más comunes: HTTP (Cl@ve) y SOAP.
- **Distribuido:** SAML no requiere una autoridad central.

Mecanismos de funcionamiento

- **Aserciones SAML.** Define esquemas XML para expresar lo que denomina “aserciones” (assertions), que son verificaciones constatadas sobre un usuario. Existen 3 tipos:
 - **Autenticaciones:** verificaciones de identidad.
 - **Autorizaciones:** las autoridades de autorización (normalmente las mismas que las de identificación) confirman o deniegan permisos de usuarios concretos ya identificados para realizar determinadas acciones.
 - **Atributos.** Expresan datos sobre una entidad (normalmente personas) que permiten a los proveedores de servicio aplicar su propia política de control de acceso o de servicio en función de la respuesta.
- **Roles.** SAML considera 3 roles:
 - **Principal:** se denomina así a la entidad que requiere ser autenticada; puede ser un usuario final u otro sistema.
 - **Proveedor de servicio (SP):** desea autenticar, autorizar o conocer algún dato de algún usuario o sistema (principal)
 - **Proveedor de identidad (IDP):** autentica, autoriza o envía datos de usuarios (principales).
- **Protocolos**

Protocolo	Función
Authentication Request Protocol	Solicitud de autenticación de un usuario

Single Logout Protocol	Define la forma de realizar un logout simultáneo de las sesiones asociadas a un usuario. El logout puede ser iniciado directamente por el usuario, o iniciado por un IDP o SP debido a la expiración por tiempo o también por decisión de un administrador
Assertion Query and Request Protocol	Solicitud de aserciones. La forma de Request de este protocolo permite solicitar a un IDP un Assertion haciendo referencia a su ID. La forma de Query define como solicitar Assertions en base al usuario y el tipo de Assertion
Name Identifier Management Protocol	Provee mecanismos para cambiar el valor o formato del identificador usado para referirse al usuario. El protocolo también define la forma de terminar una asociación de la identidad del usuario entre el IDP y el SP
Name Identifier Mapping Protocol	Provee un mecanismo para poder mapear un identificador de usuario usado entre un IDP y un SP de forma de obtener el utilizado entre el IDP y otro SP.
Artifact Resolution Protocol	Permite transportar los Assertions por referencia (a esta referencia se le denomina Artifact en el estándar) en vez de valor. Esto facilita el uso de SAML en un entorno web.

- **Bindings:** especificación que indica cómo encapsular los mensajes Request/Response del protocolo SAML en el payload del mensaje de un mecanismo de transporte concreto. SAML define numerosos bindings. Algunos de los más importantes son:
 - Binding de SAML sobre SOAP (Binding SOAP): indica cómo insertar un mensaje SAML en el cuerpo (*body*) de un mensaje SOAP con indicaciones de cómo transportar a su vez el mensaje SOAP sobreHTTP.
 - Binding de SAML sobre el método POST de HTTP. (Binding POST).
 - Binding de SAML sobre mensajes HTTP de redirección (Binding Redirección).
- **Perfiles SAML:** definen cómo combinar aserciones SAML, protocolos y bindings para poder cubrir determinados escenarios de uso y garantizar la interoperabilidad. Cabe destacar:
 - Perfil SSO para browsers (Web Browser SSO Profile). Pensado para ofrecer servicios de SSO a clientes con navegadores estándar.
 - Perfil mejorado de cliente y proxy (Enhanced Client and Proxy - ECP). Caso especial en que se emplea un mediador de proveedores de servicios de identidad (caso de Cl@ve)

SAML y Web Service-Security

Mientras que los mensajes de SAML sobre SOAP van en el cuerpo (*body*) de SOAP y forman parte de un protocolo Request/Response de autenticación, autorización o consulta de atributos; los mensajes SAML de la norma WSS son sólo aserciones que van en la cabecera (*header*) SOAP y tienen como objetivo participar de la seguridad del mensaje SOAP en sí. Normalmente son aserciones de autenticación que van firmadas mediante XML DSignture y que permiten establecer la identidad del emisor del mensaje SOAP.

4.3 JSON Web Token

JSON Web Token (JWT) es un estándar abierto que define una forma compacta y autónoma para transmitir de forma segura información entre partes como un objeto JSON. Esta información puede ser verificada y confiable porque está firmada digitalmente (puede no estarlo pero no es lo común).

Casos de uso

- **Autorización:** Este es el escenario más común para usar JWT. Una vez que el usuario ha iniciado sesión, cada solicitud posterior incluirá el JWT. Usado en SSO.

- **Intercambio de información:** los tokens web JSON son una buena manera de transmitir información de forma segura entre las partes. Debido a que los JWT se pueden firmar, por ejemplo, utilizando pares de claves públicas / privadas.

Estructura

- Cabecera consiste en dos partes: el tipo de token, que es JWT, y el algoritmo de firma que se está utilizando, como HMAC SHA256 o RSA.
- Carga útil: La segunda parte del token es la carga útil, que contiene las reclamaciones (claims). Hay tres tipos de reclamaciones:
 - Reclamaciones registradas (predefinidas)
 - Reclamaciones públicas
 - Reclamaciones privadas
- Firma: Para crear la parte de firma, debe tomar el encabezado codificado, la carga útil codificada, un secreto, el algoritmo especificado en el encabezado y firmarlo.

Forma compacta:

xxxxx.yyyyyy.zzzzz
<header(cabecera).Payload(Carga útil).Signature(firma)>

Funcionamiento

Los JWT se emiten y verifican. Cuando se emite, un servidor genera un nuevo JWT generalmente después de un proceso de inicio de sesión. Este paso implica añadir toda la información del usuario como reclamaciones JWT y firmar los datos con una clave privada. El cliente de usuario almacena este token y lo adjunta a cada solicitud de API futura.

El término usuario puede tratarse de una persona (presentando sus credenciales como usuario y contraseña, certificado digital u otros medios autorizados) o un servicio web (identificándose por ejemplo mediante certificado cualificado de sitio web, acorde al Reglamento eIDAS).

5 Seguridad de las bases de datos

Vulnerabilidades más habituales en BBDD:

- Debilidad de credenciales.
- Funciones no utilizadas.
- Inyección SQL.

Buenas prácticas:

- Mínimos privilegios.
- Elección y mantenimiento adecuados del SGBD.
- Gestión de usuarios mediante grupos.

